Database Design

Unit – 3

Topic

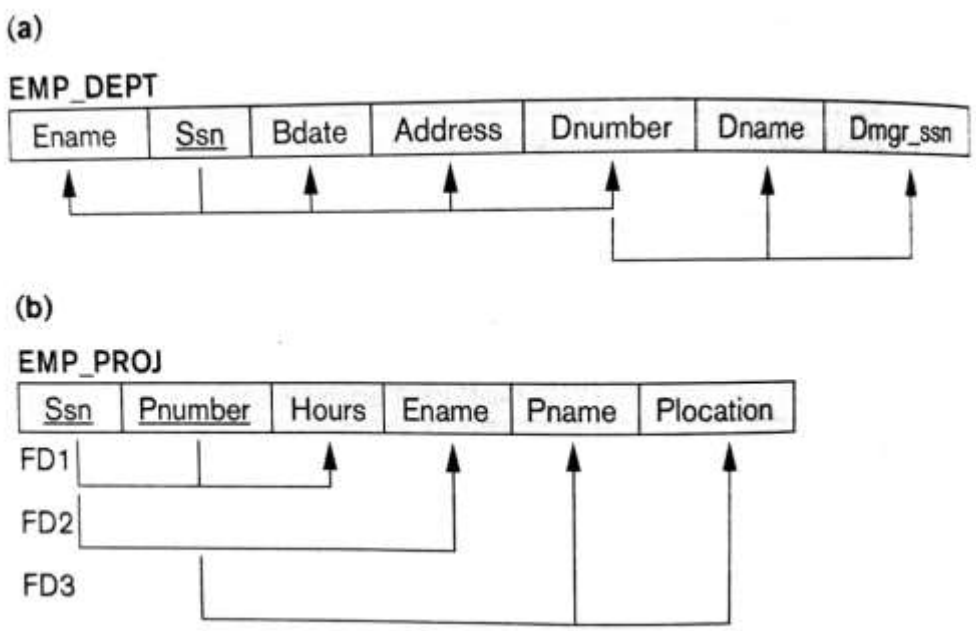
- Design guidelines for Relational Schema,
- Codd’s Rules,
- Database Relationships and Constraints – Primary Key, Foreign Key, Null, Not Null, Unique

Informal Design guidelines for Relational Schema

- Making sure that the semantics of the attributes is clear in the schema
- Reducing the redundant information in tuple
- Reducing the NULL values in the tuples
- Disallowing the possibility of generating spurious tuples

Imparting clear semantics to attributes in relations

- The semantics of a relation refers to its meaning resulting from the interpretation of attribute values in a tuple
- Guideline-1: Do not combine attributes from multiple entity types and relationship types into a single relation
- Ex. Violating guideline 1



Redundant Information in Tuples and Update Anomalies

- Insertion Anomalies: Considering Emp_Dept relation
  - Here to inert new employee tuple into the Emp_Dept relation then, we must either include the values for the department, in which the employee will work for or NULLs if not assigned any department yet
  - It is also difficult to inert a new department that has no employee. The possible option is to put NULL values in all attributes of employee
- Deletion Anomalies
- Modification Anomalies:
  - Ex. Modify the manager of a department will force to update all the employee row also who work for that department

- Guideline 2: Design the base relation schema so that no anomalies are present in the relations.

NULL values in Tuples

- Guideline 3:
- Avoid placing attributes in a base relation whose values may frequently be null.
- If NULLs are unavoidable, make sure that they apply in exceptional cases only and do not apply to majority of tuples in the relation

Generation of spurious tuples

- Guideline 4:
  - Design relational schemas so that they can be joined with equality conditions on attributes that are appropriately related pairs (PK & FK) in a way that guarantees that no spurious tuples are generated

Codd's Rules

- Every database has tables, and constraints cannot be referred to as a rational database system.
- If any database has only relational data model, it cannot be a Relational Database System (RDBMS).
- So, some rules define a database to be the correct RDBMS.
- These rules were developed by Dr. Edgar F. Codd (E.F. Codd) in 1985.
- Codd presents his 13 rules for a database to test the concept of DBMS against his relational model, and if a database follows the rule, it is called a true relational database (RDBMS).
- These 13 rules are popular in RDBMS, known as Codd's 12 rules

- Rule 0: The Foundation Rule
- The database must be in relational form. So that the system can handle the database through its relational capabilities.

- Rule 1: Information Rule
- A database contains various information, and this information must be stored in each cell of a table in the form of rows and columns.

- Rule 2: Guaranteed Access Rule
- Every single or precise data (atomic value) may be accessed logically from a relational database using the combination of primary key value, table name, and column name.

- Rule 3: Systematic Treatment of Null Values
- This rule defines the systematic treatment of Null values in database records. The null value has various meanings in the database, like missing the data, no value in a cell, inappropriate information, unknown data and the primary key should not be null.

- Rule 4: Active/Dynamic Online Catalog based on the relational model
- It represents the entire logical structure of the descriptive database that must be stored online and is known as a database dictionary. It authorizes users to access the database and implement a similar query language to access the database.

- Rule 5: Comprehensive Data SubLanguage Rule (P.T.O)

- Rule 5: The comprehensive data sublanguage rule:

- A relational system may support several languages and various modes of terminal use (for example, the fill-in-the-blanks mode). However, there must be at least one language whose statements are expressible, per some well-defined syntax, as character strings and that is comprehensive in supporting all of the following items:
- Data definition.
- View definition.
- Data manipulation (interactive and by program).
- Integrity constraints.
- Authorization.
- Transaction boundaries (begin, commit and rollback).

- Rule 6: View Updating Rule
- All views table can be theoretically updated and must be practically updated by the database systems.

- Rule 7: Relational Level Operation (High-Level Insert, Update and delete) Rule
- A database system should follow high-level relational operations such as insert, update, and delete in each level or a single row. It also supports union, intersection and minus operation in the database system.

- Rule 8: Physical Data Independence Rule
- All stored data in a database or an application must be physically independent to access the database. Each data should not depend on other data or an application. If data is updated or the physical structure of the database is changed, it will not show any effect on external applications that are accessing the data from the database.

- Rule 9: Logical Data Independence Rule
- It is similar to physical data independence. It means, if any changes occurred to the logical level (table structures), it should not affect the user's view (application). For example, suppose a table either split into two tables, or two table joins to create a single table, these changes should not be impacted on the user view application.

- Rule 10: Integrity Independence Rule
- A database must maintain integrity independence when inserting data into table's cells using the SQL query language. All entered values should not be changed or rely on any external factor or application to maintain integrity. It is also helpful in making the database-independent for each front-end application.

- Rule 11: Distribution Independence Rule
- The distribution independence rule represents a database that must work properly, even if it is stored in different locations and used by different end-users. Suppose a user accesses the database through an application; in that case, they should not be aware that another user uses particular data, and the data they always get is only located on one site. The end users can access the database, and these access data should be independent for every user to perform the SQL queries.

- Rule 12: Non Subversion Rule
- The non-submersion rule defines RDBMS as a SQL language to store and manipulate the data in the database. If a system has a low-level or separate language other than SQL to access the database system, it should not subvert or bypass integrity to transform data.

Unit-3

Cont…

Functional dependency

- A functional dependency is a constraint that specifies the relationship between two sets of attributes where one set can accurately determine the value of other sets.
- A functional dependency is denoted by an arrow "→". The functional dependency X → Y, where X is a set of attributes that is capable of determining the value of Y.
- The attribute set on the left side of the arrow, X is called Determinant, while on the right side, Y is called the Dependent.
- Functional dependencies are used to mathematically express relations among database entities and are very important to understand advanced concepts in Relational Database System

roll_no	name	dept_name	dept_building
42	abc	CO	A4
43	pqr	IT	A3
44	xyz	CO	A4
45	xyz	IT	A3
46	mno	EC	B2
47	jkl	ME	B2

- From the table we can conclude some valid functional dependencies:

  roll_no → { name, dept_name, dept_building }, Here, roll_no can determine values of fields name, dept_name and dept_building,

  roll_no → dept_name , Since, roll_no can determine whole set of {name, dept_name, dept_building}, it can determine its subset dept_name also.

  dept_name → dept_building , Dept_name can identify the dept_building accurately, since departments with different dept_name will also have a different dept_building

  More valid functional dependencies: roll_no → name, {roll_no, name} ⇢ {dept_name, dept_building}, etc.

Armstrong's axioms/properties of functional dependencies:

- Reflexivity: If Y is a subset of X, then X→Y holds by reflexivity rule
  - For example, {roll_no, name} → name is valid.
- Augmentation: If X → Y is a valid dependency, then XZ → YZ is also valid by the augmentation rule.
  - For example, If {roll_no, name} → dept_building is valid, hence {roll_no, name, dept_name} → {dept_building, dept_name} is also valid.→
- Transitivity: If X → Y and Y → Z are both valid dependencies, then X→Z is also valid by the Transitivity rule.
  - For example, roll_no → dept_name & dept_name → dept_building, then roll_no → dept_building is also valid.

Types of Functional dependencies

- Trivial functional dependency
- Non-Trivial functional dependency
- Multivalued functional dependency
- Transitive functional dependency

1. Trivial Functional Dependency

- In Trivial Functional Dependency, a dependent is always a subset of the determinant.
- i.e. If X → Y and Y is the subset of X, then it is called trivial functional dependency
- Here, {roll_no, name} → name is a trivial functional dependency, since the dependent name is a subset of determinant set {roll_no, name}
- Similarly, roll_no → roll_no is also an example of trivial functional dependency.

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

2. Non-trivial Functional Dependency

- In Non-trivial functional dependency, the dependent is strictly not a subset of the determinant.
- i.e. If X → Y and Y is not a subset of X, then it is called Non-trivial functional dependency.
- Here, roll_no → name is a non-trivial functional dependency, since the dependent name is not a subset of determinant roll_no
- Similarly, {roll_no, name} → age is also a non-trivial functional dependency, since age is not a subset of {roll_no, name}

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocation
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

3. Multivalued Functional Dependency

- In Multivalued functional dependency, entities of the dependent set are not dependent on each other.
- i.e. If a → {b, c} and there exists no functional dependency between b and c, then it is called a multivalued functional dependency.
- Ex.
- Here, roll_no → {name, age} is a multivalued functional dependency, since the dependents name & age are not dependent on each other(i.e. name → age or age → name doesn't exist !)

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18
45	abc	19

4. Transitive Functional Dependency

- In transitive functional dependency, dependent is indirectly dependent on determinant.
- i.e. If a → b & b → c, then according to axiom of transitivity, a → c. This is a transitive functional dependency

- Here, enrol_no → dept and dept → building_no,
- Hence, according to the axiom of transitivity,
- enrol_no → building_no is a valid functional dependency.
- This is an indirect functional dependency, hence called Transitive functional dependency.

enrol_no	name	dept	building_no
42	abc	CO	4
43	pqr	EC	2
44	xyz	IT	1
45	abc	EC	2

Transitive FD example

Roll_Number	Pin_Code	City_Name
011	450331	Burhanpur
012	450001	Khandwa
013	456001	Ujjain
014	452020	Indore

From above table,
Roll_Number --> Pin_Code and Pin_Code --> City_Name hold.
Than Roll_Number --> City_Name is a transitive FD.

Normalization

- First Normal Form: This rule defines that all the attributes in a relation must have atomic domains. The values in an atomic domain are indivisible units.
- Eg.

Car_model	Color	Price
1	Red, Grey, Black	5lk , 6lk , 4.5lk
2	Red, Yellow, White	5.5 lk , 4lk, 6lk
3	Black, White	7lk , 6lk
4	White, Grey	6lk , 7lk

- Convert into 1st NF as:

  {Car_Model, Color} and {Car_Model, Price}

Cont…

- Second Normal Form:
- Every non-prime attribute should be fully functionally dependent on prime key attribute.
- That is, if X → A holds, then there should not be any proper subset Y of X, for which Y → A also holds true.

Eg.

Stud_Proj: {stud_id, name, proj_id, proj_name}

Student: {stud_Id, name, proj_Id,}

Project: {proj_id, proj_name}

Cont…

- Third Normal Form:

  For a relation to be in Third Normal Form, it must be in Second Normal form and the following must satisfy

- No non-prime attribute is transitively dependent on prime key attribute.
- For any non-trivial functional dependency,

  $X \rightarrow A$, then either

  - X is a superkey or
  - A is prime attribute.

Practical

- Set Operation: Clause
  - UNION,
  - UNION ALL,
  - INTERSECTION,
  - MINUS

Union

- Multiple queries can be put together using UNION clause
- For using UNION clause no. of columns and it’s datatype must be same
- The IN operator has a higher precedence than the UNION operator
- NULL Values are not ignored during duplicate checking
- Eg.

```
select a_id, city from applicant where city like 'Baroda'
union
select e.et_id, d.city from entrance_test e, etest_details
where e.et_id = d.et_id and d.city like'Baroda'
```

Intersect

- Displays only common rows as output from multiple table's intersection
- Intersect does not ignore NULL values
- Eg.

Suppose need to display Enroll_No and Name of students who are the 1st year and 2nd year rankers.

- Rankers are stored in separate tables of 1st year and 2nd Year
- So can use Intersect clause as:

```
Select Enroll_No, Name from FYRank
INTERSECT
Select Enroll_No, Name from SYRank
```

Minus

- The output of Minus clause are the rows from the output of 1st query after filtering the rows received by the second query

- Eg.
- Display students who are ranker I 1st year but not in 2nd year

```
Select Enroll_No, Name from FYRank
MINUS
Select Enroll_No, Name from SYRank
```

Unit-3

Cont…

Recap….

- Normalization:
  - The process of decomposing unsatisfactory "bad“ relations by breaking up their attributes into smaller relations
  - Normalization is carried out in practice so that the resulting designs are of high quality and meet the desirable properties

- Normal form:
  - Condition using keys and FDs of a relation to certify whether a relation schema is in a particular normal form

Definitions of Keys and Attributes Participating in Keys

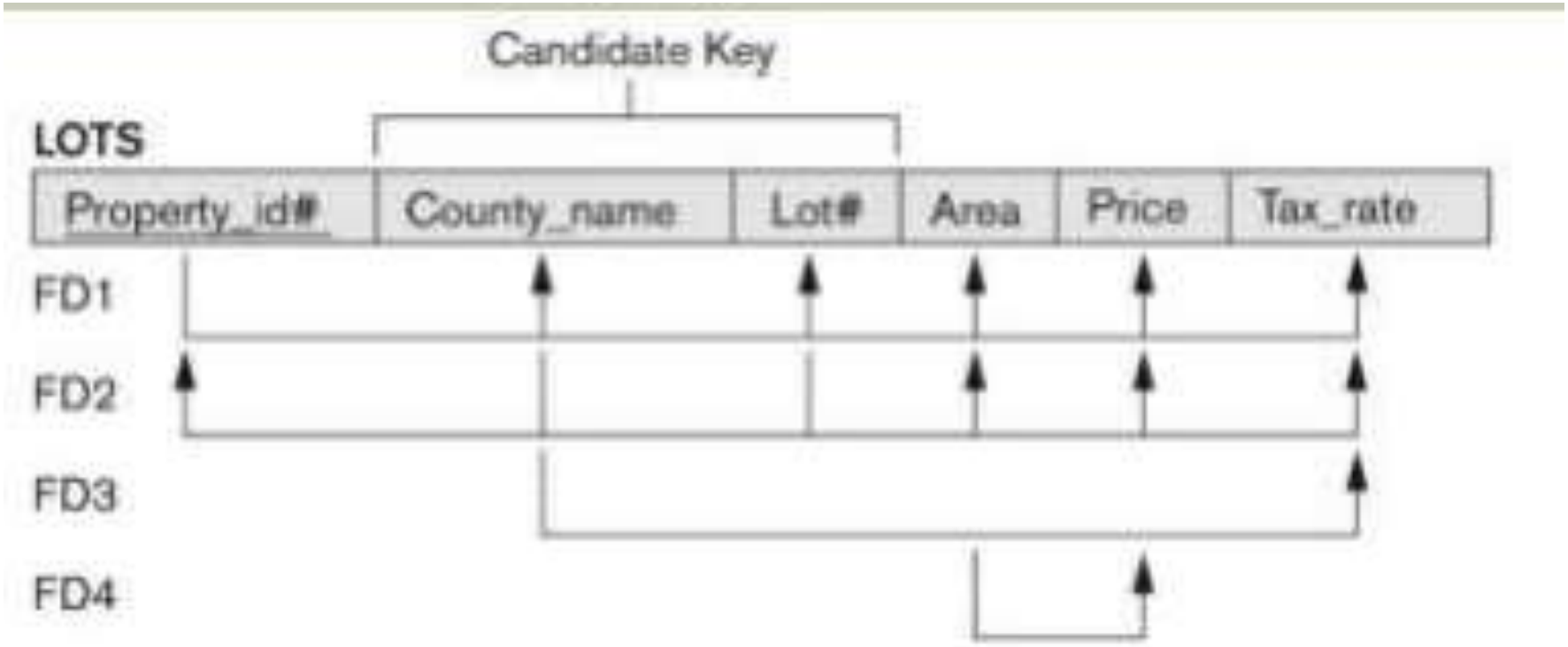
- A super key of a relation schema R = {A1, A2, ..., An} is a set of attributes S subset-of R with the property that no two tuples t1 and t2 in any legal relation state r of R will have t1[S] = t2[S]
- A Candidate key K is a super key with the additional property that removal of any attribute from K will cause K not to be a super key any more.

OR

- Minimal Super Key with unique and not null values can be a candidate key
- One of the candidate keys is arbitrarily designated to be the primary key, and the others are called secondary Keys / Alternate Keys

- A Prime attribute must be a member of some candidate key
- A Nonprime attribute is not a prime attribute— that is, it is not a member of any candidate key

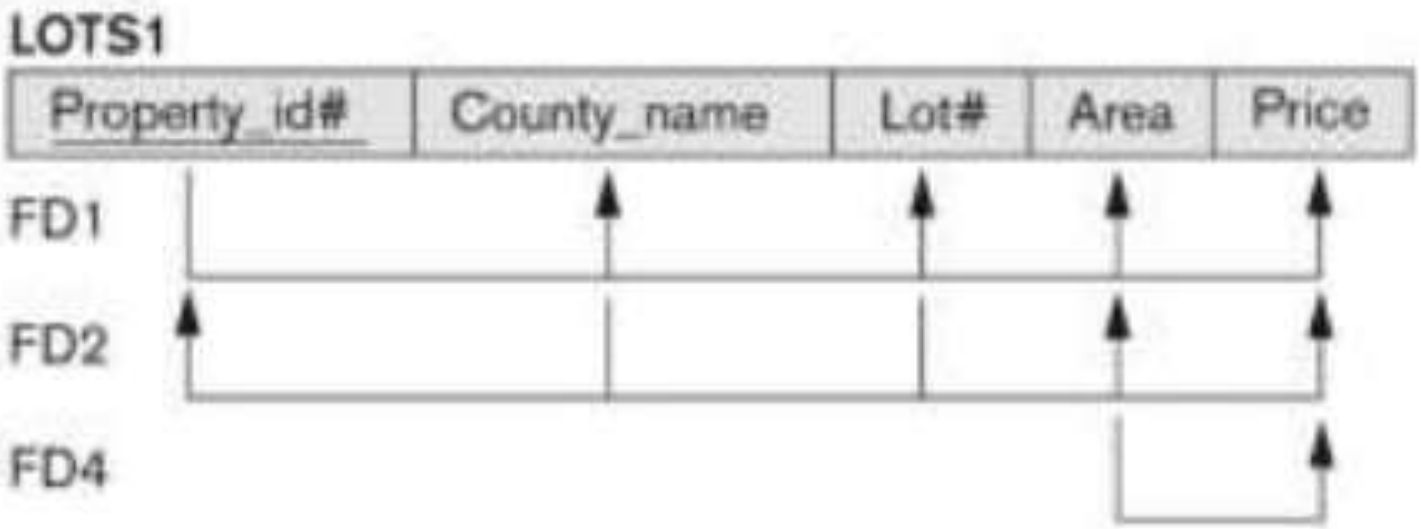
Example for Normalization

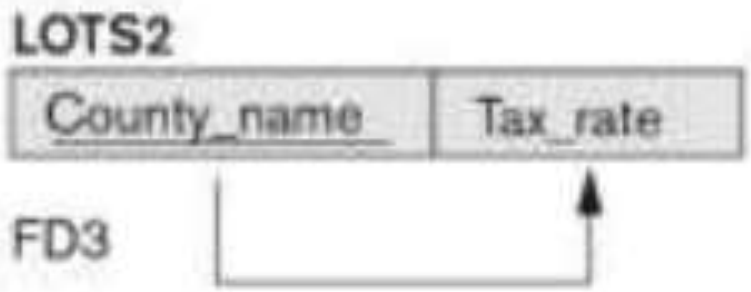


- The above relation describes parcels of land for sale in various county of a state. Suppose there are 2 candidate keys as:
- 1. Property_ID
- 2. County_Name and Lot_No

2 NF Conversion

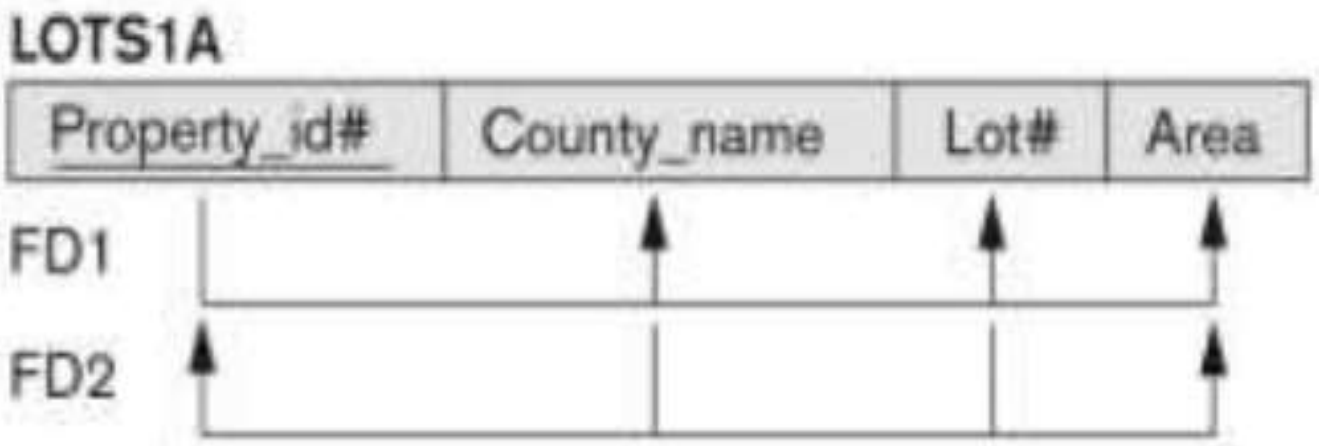
- Remove partial functional dependency
- County_Name -> Tax_Rate

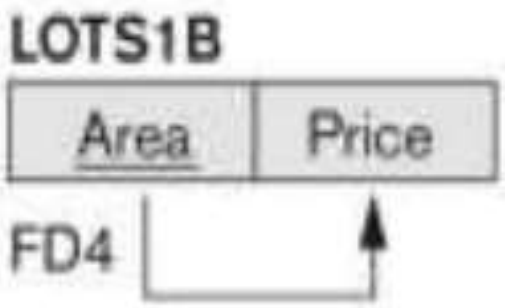




3 NF Conversion of relation LOT1

- Here remove transitive dependency
- Area -> Price and
- Property_Id -> Area





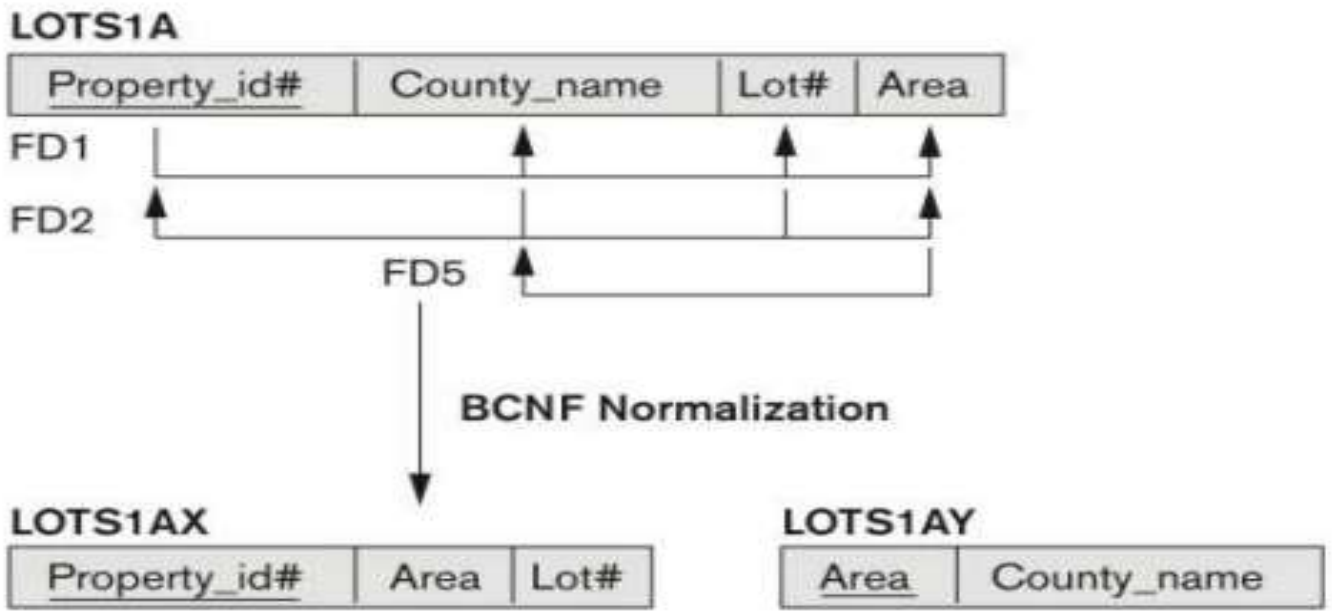
BCNF – Boyce Codd Normal Form

- BCNF is the advance version of 3NF. It is stricter than 3NF.
- A table is in BCNF if every functional dependency X → Y,
  - **X is the super key of the table**.
- For BCNF, the table should be in 3NF, and for every FD, LHS is super key.

- General definition of 3NF is:
  - For any FD X->Y:
    I. X is a super Key of R OR
    II. Y is a Prime Attribute of R

BCNF

- In the above relation LOTs1A if we assume that the Lot size/ area is fix per county and they are from 2 county only
- So in that case we can add one more FD i.e. Area -> County_Name
- So, according to 3NF this is allowed FD as: Area is not a super key according to rule (i) but as rule (ii) says County_Name is a prime attribute.
- But according to BCNF only one condition is allowed for FD is that the determinant must be a super key. So, Need to convert the relation LOTs1A into BCNF as shown below:



BCNF Example

- Candidate Key:
- {Student,Course}

- FDs:
- {Student,Course} -> Instructor
- Instructor -> Course

- Decomposition Rule:

$((R1 \cap R2) \rightarrow (R1 - R2))$ is in F^+, or
$((R1 \cap R2) \rightarrow (R2 - R1))$ is in F^+.

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

Conversion into BCNF

D1: {student, instructor} and {student, course}

D2: {course, instructor } and {course, student}

D3: {instructor, course } and {instructor, student} ✓

- Out of all above 3 decomposition
- Only D3 follows the decomposition rule for BCNF
- So D3 is the only valid decomposition

Examples for Normalization

R = {Course_No, Sec_No, Off_Dept, Credit_Hrs, Course_Level, Inst_SSN,
sem, year, Days_Hrs, Room_No, No_of_Stud}

Functional Dependencies:

{Course_No} -> {Off_Dept, Credit_Hrs, Course_Level}

{Course_No, Sec_No, Sem, Year} -> {Days_Hrs, Room_No, No_of_Stud, Inst_SSN}

{Room_No, Days_Hrs, Sem, Year} -> {Inst_SSN, Course_No, Sec_No}

- **2NF:**
- Course_Master:

Course_No	Off_Dept	Course_Hrs	Course_Level

- Course_Taken:

Course_No	Sec_No	Sem	Year	Days_Hrs	Room_No	Inst_SSN	No_of_Stud

- 3NF
- No Transitive Dependancies
- **BCNF:** Decomposition of course taken as below – (use 3rd FD for the same)
- Course_Details

Course_No	Sec_No	Sem	Year	No_of_Stud

- Instructor_Load

Sem	Year	Room_No	Days_Hrs	Inst_SSN

Course_No	Sec_No	Room_No	Days_Hrs

VIEW

- To reduce redundant data to the minimum possible, to prevent all users from accessing all columns of a table and for data security reason, Oracle allows the creation of an object called a View
- A view is mapped to a select sentence.
- A table on which view is based is described in the from clause of the SELECT statement
- It is stored only as a definition in Oracle’s system catalog
- When a reference is made to a view, its definition is scanned, the base table is opened and a view is created on the top of the base table.

Syntax

Create view <View_Name>

AS Select <list of col_name> from table

Where col_name = experience

Group by <criteria> having <predicate>

NOTE:

1. “ORDER BY” clause can’t be used while creating a View
2. The column of the table are related to the view using One-to-One relationship

Operations can be performed on VIEW

- Renaming the columns of a view

  Create view V1 AS Select col_1 “c1”, col_2 “c2” from table
- Selecting data set from the view

  Select c1, c2 from v1;
- With specific conditions, a view can be updated
  - View defined from single table
  - For inserting record in the View, it must include Primary Key and NOT NULL columns of the base table
  - For deleting or updating record in the view, it can **Exclude** Primary Key and NOT NULL columns of the base table

Unit - 3

Cont…

Normalisation Example - 1

Dept_Name	D_No	D_Mgr_SSN	Dept_Location
Research	5	501	Banglore, Kolkota
Administration	4	401	Delhi, Mumbai
Sales	1	101	Ahmedabad, Banglore, Delhi

Example - 2

SSN	Proj_No	Emp_Hours	Emp_Name	Proj_Name	Proj_Loc
101	P1	20	Ankit Shah	Renovation	Bopal
101	P2	50	Ankit Shah	Construction	Ghuma
102	P2	70	Ankit Shah	Construction	Bopal
103	P2	40	Bhumi Patel	Construction	Sarkhej
103	P3	30	Bhumi Patel	Designing	Maninagar

Example - 3

Property_ID	City	Plot_No	Area	Price

Functional Dependencies are given as below:

FD1: Property_ID → {City, Plot_No, Area, Price}

FD2: {City, Plot_No} → {Property_ID, Area, Price}

FD3: Area → Price

FD4: Area → City (Consider this special condition, FD just for BCNF)

PL / SQL

- It is a development tool
- Supports SQL manipulation and also provide facilities of condition checking, branching and looping
- Entire PL/SQL block is sent all in one to the Oracle engine for execution
- It permits error handling
- Variables can be declared and values can be stored inside that variable using user input as well as from SQL statement
- Applications written in PLSQL are portable to any computer h/w and s/w
- PL/SQL engine resides inside the oracle engine

Unit-3

Cont…

Multivalued Dependency

- Multivalued dependency occurs when two attributes in a table are independent of each other but, both depend on a third attribute.
- A multivalued dependency consists of at least two attributes that are dependent on a third attribute that's why it always requires at least three attributes.

Example: Suppose there is a bike manufacturer company which produces two colors(white and black) of each model every year.

BIKE_MODEL	MANUF_YEAR	COLOR
M2011	2008	White
M2001	2008	Black
M3001	2013	White
M3001	2013	Black
M4006	2017	White
M4006	2017	Black

Here columns COLOR and MANUF_YEAR are dependent on BIKE_MODEL and independent of each other.
In this case, these two columns can be called as multivalued dependent on BIKE_MODEL. The representation of these dependencies is shown below:

1.BIKE_MODEL → → MANUF_YEAR

4NF

- Any relation is said to be in the fourth normal form when it satisfies the following conditions:
  - It must be in Boyce Codd Normal Form (BCNF).
  - It should have no multi-valued dependency.

STUDENT

STU_ID	COURSE	HOBBY
21	Computer	Dancing
21	Math	Singing
34	Chemistry	Dancing
74	Biology	Cricket
59	Physics	Hockey

Join Dependency

- Join decomposition is a further generalization of Multivalued dependencies. If the join of R1 and R2 over C is equal to relation R then we can say that a join dependency (JD) exists, where R1 and R2 are the decomposition R1(A, B, C) and R2(C, D) of a given relations R (A, B, C, D).
- Alternatively, R1 and R2 are a lossless decomposition of R. A JD ⋈ {R1, R2, …, Rn} is said to hold over a relation R if R1, R2, ….., Rn is a lossless-join decomposition.
- The *(A, B, C, D), (C, D) will be a JD of R if the join of join's attribute is equal to the relation R.
- Here, *(R1, R2, R3) is used to indicate that relation R1, R2, R3 and so on are a JD of R.

5NF

- A relation is in Fifth Normal Form (5NF), if it is in 4NF, and won't have join dependency.
- You can also consider that a relation is in 5NF, if the candidate key implies every join dependency in it.

A relation R is in 5NF if and only if every join dependency in R is implied by the candidate keys of R. A relation decomposed into two relations must have loss-less join Property, which ensures that no spurious or extra tuples are generated, when relations are reunited through a natural join.

Example

The below relation violates the Fifth Normal Form (5NF) of Normalization –

<Employee>

EmpName	**EmpSkills**	**EmpJob** (Assigned Work)
David	Java	E145
John	JavaScript	E146
Jamie	jQuery	E146
Emma	Java	E147

The above relation can be decomposed into the following three tables; therefore, it is not in 5NF –

<EmployeeSkills>

EmpName	EmpSkills
David	Java
John	JavaScript
Jamie	jQuery
Emma	Java

The following is the <EmployeeJob> relation that displays the jobs assigned to each employee –

<EmployeeJob>

EmpName	EmpJob
David	E145
John	E146
Jamie	E146
Emma	E147

Here is the skills that are related to the assigned jobs –

<JobSkills>

EmpSkills	EmpJob
Java	E145
JavaScript	E146
jQuery	E146
Java	E147

Example 2

A relation R satisfies join dependency if R is equal to the join of R1,R2,.....Rn where Ri is a subset of the set of attributes of R.

Relation R

Dept	Subject	Name
CSE	C	Ammu
CSE	C	Amar
CSE	Java	Amar
IT	C	bhanu

The above relation says that dept offers many elective subjects which are taken by a variety of students. Students have the opinion to choose subjects. Therefore all three fields are needed to represent the information.

The above relation does not show non-trivial MVDs since the attributes subject and name are dependent; they are related to each other (A FD subject->name exists). The relation cannot be decomposed in two relations (dept, subject) and (dept,sname).

Therefore the relation can be decomposed into following three relations –

R1(dept, subject)

R2(dept, name) and

R3(subject, name) and it can be shown that decomposition is lossless.

R1

Dept	Subject
CSE	C
CSE	Java
IT	C

R2

Dept	Name
CSE	Ammu
CSE	Amar
IT	bhanu

R3

Subject	Name
C	Ammu
C	Amar
Java	Amar
C	bhanu

EXAMPLE - 3

Properties – A relation R is in 5NF if and only if it satisfies following conditions:

1. R should be already in 4NF.
2. It cannot be further non loss decomposed (join dependency)

Example – Consider the above schema, with a case as "if a company makes a product and an agent is an agent for that company, then he always sells that product for the company". Under these circumstances, the ACP table is shown as:

Table – ACP

AGENT	COMPANY	PRODUCT
A1	PQR	Nut
A1	PQR	Bolt
A1	XYZ	Nut
A1	XYZ	Bolt
A2	PQR	Nut

Table – R1

AGENT	COMPANY
A1	PQR
A1	XYZ
A2	PQR

Table – R2

AGENT	PRODUCT
A1	Nut
A1	Bolt
A2	Nut

Table – R3

COMPANY	PRODUCT
PQR	Nut
PQR	Bolt
XYZ	Nut
XYZ	Bolt

Result of Natural Join of R1 and R3 over 'Company' and then Natural Join of R13 and R2 over 'Agent'and 'Product' will be table **ACP**.

Hence, in this example, all the redundancies are eliminated, and the decomposition of ACP is a lossless join decomposition. Therefore, the relation is in 5NF as it does not violate the property of lossless join.

FAQs

What is join dependency? Give examples.

Whenever we can recreate a table by simply joining various tables where each of these tables consists of a subset of the table's attribute, then this table is known as a Join Dependency.
Thus, it is like a generalization of MVD. We can relate the JD to 5NF. Herein, a relation can be in 5NF only when it's already in the 4NF. Remember that it cannot be further decomposed.

What is multivalued and join dependency in DBMS?

A multivalued dependency is a case in which a determinant gets associated with some set of values. Whenever isolated, the MVD does not have modification anomalies.
Join dependency or JD is a constraint that is similar to FD (functional dependency) or MVD (multivalued dependency). JD is satisfied only when the concerned relation is a join of a specific number of projections. Thus, such a type of constraint is known as a join dependency.

What is join dependency in 5NF?

Some relations can be in 5NF whenever it satisfies the 4NF and there exists no join dependency. Thus, a relation can have join dependency in case it can be recreated when we join multiple sub relations, while each of the sub relations contains a subset of the original relation's attributes.

What is a multivalued dependency, and how do they affect the normalization process?

A multivalued dependency is a case in which a determinant gets associated with some set of values. Whenever isolated, the MVD does not have modification anomalies. Thus, the tables that have these isolated dependencies can be considered to be in the 4NF (fourth normal form).

PL / SQL

- It is a development tool
- Supports SQL manipulation and also provide facilities of condition checking, branching and looping
- Entire PL/SQL block is sent all in one to the Oracle engine for execution
- It permits error handling
- Variables can be declared and values can be stored inside that variable using user input as well as from SQL statement
- Applications written in PLSQL are portable to any computer h/w and s/w
- PL/SQL engine resides inside the oracle engine

PL/SQL

- **Generic PL/SQL block:**
- Declare Section
  - Here, memory variable an other Oracle objects can be declared, initialized if required. It’s an optional block
- Begin Section
  - It consists of a set of SQL and PL/SQL statements, which describe process that have to be applied to table data.
- Exception Section
  - This section deals with handling of errors that arise during execution of the data manipulation statements, by PLSQL code block
- End Section
  - It marks the end of the PLSQL block.

PLSQL Character Set

- Every database character set includes these basic characters:

  1. Latin letters: A through Z and a through z
  2. Decimal digits: 0 through 9
  3. Punctuation characters like: () + - * / < > = ! , : . ‘ @ % “ # $ ^ & _ \ { } ? ||
  4. Whitespace characters: space, tab, new line, and carriage return

Control Structure

- Condition Control ---- IF – ELSEIF – ELSE – END IF
- Iterative Control ----- While Loop, FOR Loop
- Sequential Control ----GOTO Statement

Syntax

```
IF  <condition>  then
        <Action>
ELSIF  <Condition> then
        <Action>
ELSE
        <Action>
END IF;
```

Iterative Control - Syntax

```
LOOP
        <Sequence of Statements>
        <increment or decrement of variable>

        EXIT WHEN <Condition>
END LOOP;
```

```
WHILE  <Condition>
LOOP
        <Sequence of Statements>
        <increment or decrement of variable>

END LOOP;
```

Syntax

```
FOR variable in [Reverse] start .. End
        LOOP
                <Action>
        END Loop;
```

- The variable in the For Loop need not be declared. Also the increment value cannot be specified.
- The FOR Loop Variable is always incremented by 1